Независимые случайные потоки I

Для решения многих задач приходится получать достаточно большую выборку значений

$$x_1, \quad x_2, \quad \ldots, \quad x_N$$

некоторой случайной величины $\boldsymbol{X}$. Предполагается, что выборочные значения являются независимыми в вероятностном смысле.

Часто для получения одного значения x_i величины $\boldsymbol{X}$ используется несколько псевдослучайных чисел из последовательности

$$U_1, \quad U_2, \quad \ldots .$$

В случае распределения получения выборочных значений x_1, x_2, ..., x_N между несколькими вычислительными

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки II

процессами надо по-прежнему обеспечить их статистическую независимость.

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки III

Необходимым условием для этого является использование различных частей последовательности

$$U_1, \quad U_2, \quad \ldots, \quad U_n, \quad \ldots$$

или даже различных последовательностей

$$\begin{array}{cccc} U'_1, & U'_2, \ldots, & U'_n, & \ldots \\ U''_1, & U''_2, & \ldots, & U''_n, \quad \ldots \end{array}$$

разными вычислительными процессами. В этом случае говорят, что необходимо получить достаточное число *независимых потоков* псевдослучайных чисел.

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки IV

Почему плохо использовать перекрывающиеся участки последовательности U_1, U_2, ... для генерации x_1, x_2, ..., x_N?

1. работа разных вычислительных процессов над одним потоком данных даст идентичные результаты — растрата ресурсов!!!
2. наличие одинаковых выборочных значений может быть ложно интерпретировано как наличие малой дисперсии величины $\boldsymbol{X}$ и, как следствие, точность полученного результата окажется завышенной.

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки V

Метод непересекающихся отрезков.

Пусть заранее известно количество N псевдослучайных чисел, необходимое для работы одного вычислительного процесса.
Пусть предполагаемое число процессов L и период генератора d удовлетворяют условию

$$L \cdot N \leq d.$$

Разобьем период U_1, U_2, ..., U_d на *непересекающиеся отрезки*

$$(U_1, U_2, \ldots, U_N), \quad (U_{N+1}, U_{N+2}, \ldots, U_{2N}), \quad \ldots$$

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки VI

или

$$(U_1, U_2, \ldots, U_N), \quad (U_{[d/L+1]}, U_{[d/LN+2]}, \ldots, U_{[d/L]+N}), \quad \ldots$$

Основная трудность — быстрый перехо к значениям U_{N+1}, U_{2N+1}, ... без вычисления промежуточных значений. Например, возьмем мультипликативный конгруэнтный генератор. Тогда

$$U_{N+1} = a^N U_1 \mod m, \quad U_{2N+1} = a^N U_{N+1} \mod m, \quad \ldots .$$

Известно, что возведение в целую степень N можно выполнить за число операций порядка $O(\log N)$. Вот *бинарный метод* возведения в степень:

$$a^N = \begin{cases} (a^{N/2})^2 & \text{при четном } N, \\ a(a^{(N-1)/2})^2 & \text{при нечетном } N. \end{cases}$$

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки VII

Существуют и другие методы быстрого возведения в целую степень.

Listing 9: Функция быстрого возведения в степень

```
int intpow(int a, unsigned n, unsigned m){
  int ans;
  if(0 != n % 2) ans = a % m; else ans = 1;
  for(;;){
    n /= 2;
    if (0 == n) break;
    a = a * a % m;
    if (0 != n % 2) ans = a * ans % m;
  };
return ans;
};
```

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки VIII

В общем случае, с математической точки зрения, каждый из L вычислительных процессов работает с псевдослучайной последовательностью, имеющей в качестве параметра начало последовательности.

Метод подпоследовательностей.

Для каждого из L вычислительных процессов требуется набор из N псевдослучайных чисел, и имеется генератор с периодом $d \geq L \cdot N$. Тогда каждому вычислительному процессу выделяется ровно одна из подпоследовательностей

$$\{U_1, U_{L+1}, U_{2L+1}, \ldots\}; \quad \{U_2, U_{L+2}, U_{2L+2}, \ldots\}; \quad \ldots$$
$$\ldots; \{U_L, U_{2L}, U_{3L}, \ldots\}.$$

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки IX

Например, выше была приведена формула для линейного конгруэнтного генератора (стр. 56). По сравнению с методом непересекающихся отрезков, данный метод возможно использовать и когда заранее неизвестно требуемое количество N псевдослучайных чисел для каждого из вычислительных процессов.

Идея выделения подпоследовательностей легко модифицируется под запросы конкретной задачи. Предположим, что для получения одного значения случайной величины $\boldsymbol{X}$ используются два псевдослучайных числа. Последовательная программа будет работать с парами чисел

$$(U_1, U_2), (U_3, U_4), \ldots .$$

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки X

Чтобы параллельные вычислительные процессы работали с точными подпоследовательностями исходной последовательности, следует воспользоваться следующим разбиением:

$$\{U_1, U_2, U_{2L+1}, U_{2L+2}, \ldots\}; \quad \{U_3, U_4, U_{2L+3}, U_{2L+4}, \ldots\}; \quad \ldots$$
$$\ldots; \{U_{2L-1}, U_{2L}, U_{4L-1}, U_{4L}, \ldots\}.$$

В любом случае при использовании данного метода надо проверять, что подпоследовательности в результате снова удовлетворяют таким требованиям, как равномерность распределения, некоррелированность, и т. д.

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки XI

Параметризация итерационной процедуры.

Многие генераторы псевдослучайных чисел содержат набор параметров. Присвоим каждому вычислительному процессу свои значения параметров.

- Разные вычислительные процессы будут работать с принципиально различными псевдослучайными последовательностями, а не с различными частями одной и той же последовательности.
- Преимуществом такого подхода является то, что не происходит уменьшения периода порождаемой последовательности в L раз.

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки XII

Пример: линейный конгруэнтный генератор.

Параметризация приводит к системе рекуррентных соотношений

$$X_{n+1,j} = a_j X_{n,j} + c_j \mod m, \quad n = 0, 1, \ldots; l = 1, 2, \ldots, L.$$

Индекс j — номер потока псевдослучайных чисел.

Известны следующие рекомендации.

1. При модуле $m = 2^p$ исследователями были изучены генераторы с $a_j \equiv a$, и попарно взаимно простыми аддитивными константами c_j. В частности, рекомендуется выбирать в качестве констант последовательные простые числа, не превосходящие $\sqrt{m/2}$. При такой параметризации статистическая корреляция между разными потоками оказывается допустимо малой.

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки XIII

2. Модуль m является простым числом, возможно, близким к числу вида 2^p. В этом случае предлагается использовать в качестве параметра сомножитель a_j. В частности, полагают $a_j = a^{\ell_j}$, где ℓ_j — j-е целое число, взаимно простое с $m-1$.

Пример: явный обратный конгруэнтный генератор (explicit inverse congruent generator, EICG).

$$X_n = \overline{an+b} \mod m, \quad n = 1, 2, \ldots . \tag{8}$$

Этот генератор дает явную формулу для вычисления всех членов последовательности вместо рекуррентных формул. К нему, очевидно, применимы все приведенные ранее

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки XIV

методы. Рассмотрим следующую параметризацию генератора (8):

$$X_{n,j} = \overline{a_j n + c_j} \mod m, \quad n = 1, 2, \ldots; j = 1, 2, \ldots, N. \quad (9)$$

Известно, что хорошие статистические свойства векторов

$$(X_{n,1}, X_{n,2}, \ldots, X_{n,L})$$

имеют место, если все числа вида $c_j \overline{a_j}$ различны. В частности, можно положить $a_j \equiv a$, $c_j = a(j-1) + c \mod m$, тогда

$$c_j \overline{a_j} = (j-1) + c\overline{a} \mod m.$$

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки XV

Listing 10: Явный обратный конгруэнтный генератор с поддержкой нескольких потоков псевдослучайных чисел

```
// seicgrng.h - definition of the generator
//     x[n,j] = inv(a[j] * x[n-1,j] + b[j]) mod m
#ifndef SEICGRNG_H
#define SEICGRNG_H

#include <cmath>
#include <vector>

using namespace std;

class seicgrng{
  vector<unsigned> seed;
  unsigned a, c;
  seicgrng();
```

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки XVI

```
 public:
  seicgrng(unsigned a, unsigned c,
    unsigned _seed, int L);
  float next(int snum);
};

#endif // SEICRNG_H

// seicgrng.cc - implementation
#include "seicgrng.h"
#include "vectypes.h"
#include <stdio.h>
#include <algorithm>

// a*(a^-1)+2147483647*b=1;
inline int invmod(unsigned a){
```

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки XVII

```
  i4vec u={1, 0, a, 0},
        v = {0, 1, (unsigned)2147483647, 0}, t;
  int q;
  while(v.i[2]>0){
    q = u.i[2] / v.i[2];
    t.v = u.v - (v.v * (v4si){q, q, q, q});
    u.v = v.v;
    v.v = t.v;
  };
  if (u.i[0]>=0) return u.i[0];
  else {
    q = u.i[0] % 2147483647;
    return 2147483647+q;
  }
};
```

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки XVIII

```
seicgrng::seicgrng(unsigned _a, unsigned _c,
                   unsigned _seed, int L)
    :a (_a), c(_c)
{
  seed.resize(L);
  fill(seed.begin(),seed.end(),
       _seed%2147483647);
};

float seicgrng::next(int snum){
  float ans =
     invmod(seed[snum]*a+a*snum+c)
       /2147483647.0;
  ++seed[snum];
  return ans;
}
```

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки XIX

Параметризация циклов.

Для некоторых известных генераторов максимально допустимая длина цикла существенно меньше числа возможных состояний «внутренней памяти» генератора. Тогда все возможные состояния внутренней памяти образуют несообщающиеся классы циклических состояний.

Поэтому возможно выделять каждому вычислительному процессу свою уникальную псевдослучайную последовательность максимально возможного периода.

Пример: генератор Фибоначчи Генератор, задаваемый рекуррентным соотношением

$$X_n = X_{n-i} + X_{n-j} \mod 2^e, \quad i < j,$$

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки XX

имеет теоретически период $2^{e-1}(2^j - 1)$.
Для его работы необходимо хранить j значений X_{n-1}, X_{n-2}, $\ldots$, X_{n-j}. Пусть каждое значение занимает e бит. Тогда общее число возможных состояний генератора равно $2^{ej} - 1$ (исключается нулевое состояние).
Полный период дают начальные состояния, содержащие хотя бы одно нечетное число.
Итак, остается всего $(2^j - 1)2^{j(e-1)}$ допустимых состояний генератора. Общее число возможных циклов есть

$$\frac{(2^j - 1)2^{j(e-1)}}{(2^j - 1)2^{e-1}} = 2^{(j-1)(e-1)}.$$

Таким образом, для получения параллельных потоков псевдослучайных чисел с помощью генераторов вида (6)

© А.В. Зорин, ННГУ, 2021

Независимые случайные потоки XXI

необходимо уметь выбирать начальные значения, принадлежащие различным циклическим классам. Точный алгоритм выбора начальных состояний из разных классов приведен в статье

- A fast, high-quality, and reproducible lagged-fibonacci pseudorandom number generator / M. Mascagni, S. A. Cuccaro, D. V. Pryor, M. L. Robinson // J. Comput. Physics, 1995. — No. 15. — P. 211–219.

© А.В. Зорин, ННГУ, 2021

Некоторые полезные ресурсы I

- Специализированная библиотека SPRNG http://sprng.cs.fsu.edu/ — независимые потоки псевдослучайных чисел в паралалельных программах на системах с разделенной памятью (MPI). Один поток на процессор. Набор генераторов: линейный конгруэнтный, мультипликативный генератор Фибоначчи с запаздыванием и др. В основе разделения принцип параметризации генераторов.
- Библиотека RngStreams http://www.iro.umontreal.ca/~{}lecuyer/myftp/streams00/ может быть использована как совместно с MPI, так и в многопоточных приложениях с OpenMP. В основе создания нескольких потоков здесь лежит разделение на подпоследовательности.

© А.В. Зорин, ННГУ, 2021

Некоторые полезные ресурсы II

- Библиотека `RNG`
  `http://www.iro.umontreal.ca/~lecuyer`
  объектно-ориентированная библиотека длинных псевдослучайных последовательностей и подпоследовательностей (по-видимому, пригодна для многопоточных приложений с общей памятью).

© А.В. Зорин, ННГУ, 2021

Задания для самостоятельной работы

1. Реализовать параллельный метод вычисления объема сферы методом отрезков и методом подпоследовательностей с использованием OpenMP.

© А.В. Зорин, ННГУ, 2021